03CIT4067 Cybersecurity Project Proposal

Web Application Security Risk Detection & Remediation Demonstration System

A Hybrid Approach Combining OWASP Juice Shop and Custom Flask Modules for Modern Vulnerability Mitigation

TEAM MEMBERS (GROUP 7)

WONG Lai Man (Leader) | KHAN MUHAMMAD MUSTAFA | ZOUTAO

SUPERVISOR

Dr. LIAM XU

PROBLEM STATEMENT

Identified Issues & The Industry Gap

Section 2

1. Injection & Misconfig

Persistent Threats: A05 vulnerabilities dominate modern attack vectors, exposing auth controls and sensitive data.

2. Fail-Open Flaws

A10 Exceptional Handling: Systemic operational flaw where unexpected runtime errors default to granting admin privileges.

3. The Critical Gap

Missing Code Demos: DAST scanners flag risks but lack integrated, step-by-step code remediation guidance for developers.

PROJECT VISION

Goals, SMART Objectives & Impact

CORE VISION STATEMENT

Design, deploy, and evaluate a hybrid web security system detecting, analyzing, and remediating OWASP Top 10: 2025 risks.

SMART Objectives

- **W9 Term 1:** Deploy Juice Shop Docker & Flask A10 Module.
- **Term 2 Audit:** Eliminate High/Medium CSP/CORS ZAP alerts.
- **Deliverables:** Report, STRIDE analysis & 15-min live demo.

Significance of Study

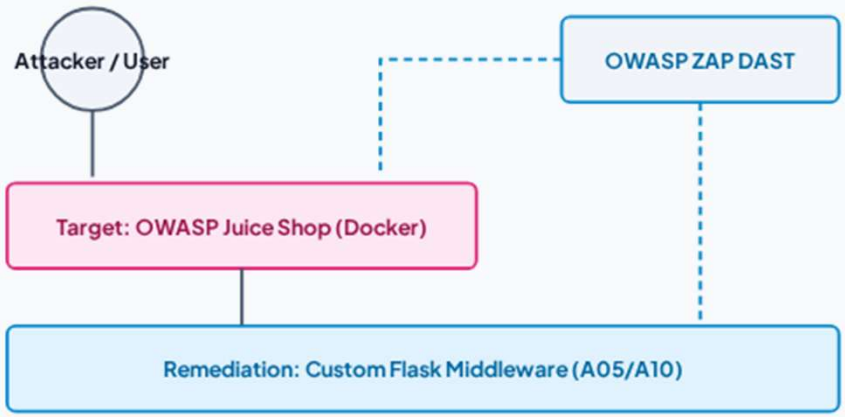
Operationalizes 2025 OWASP updates into an interactive lab environment. Directly benefits junior engineers with side-by-side comparative code defense.

TECHNICAL FOUNDATION

Hybrid System Architecture & Tech Stack

Section 5

FIGURE 1: ARCHITECTURE TOPOLOGY



OWASP Juice Shop & Docker
Comprehensive vulnerable web app isolated via containerization.

Python Flask Middleware
Lightweight custom backend enforcing headers and fail-closed logging.

OWASP ZAP Scanner
DAST scanning tool providing pre/post remediation risk metrics.

DAST EVALUATION

OWASP ZAP Baseline Security Audit

DAST Baseline



⚠Primary Medium Alerts Identified

- **Missing CSP Header:** Vulnerable to XSS and data injection attacks.
- **CORS Misconfiguration:** Permissive cross-domain data disclosure.

Targeted Remediation Strategy

Focus custom Flask middleware remediation specifically on hardening A05 security headers and enforcing A10 fail-closed exception handling.

CODE-LEVEL REMEDIATION

A05: Security Misconfiguration Defense

Remediation 1

VULNERABLE CODE

app.config['DEBUG'] = True @app.route('/api/data') def get_data(): # Exposes debug trace # Missing Security Headers return jsonify(data)

app.config['DEBUG'] = True @app.route('/api/data') def get_data(): # Exposes debug trace # Missing Security Headers return jsonify(data)

REMEDIATED CODE

app.config['DEBUG'] = False @app.after_request def add_security_headers(resp): resp.headers['X-Frame-Options'] = 'DENY' resp.headers['Content-Security-Policy'] = "default-src 'self'" return resp

app.config['DEBUG'] = False @app.after_request def add_security_headers(resp): resp.headers['X-Frame-Options'] = 'DENY' resp.headers['Content-Security-Policy'] = "default-src 'self'" return resp

CODE-LEVEL REMEDIATION

A10: Fail-Open vs. Fail-Closed Architecture

Remediation 2

FAIL-CLOSED MIDDLEWARE IMPLEMENTATION

```
try: verify_user_session(request) except Exception as e: # Log structured critical audit event
logger.critical(json.dumps({"event": "AUTH_FAIL", "error": str(e)})) # Enforce strict Fail-Closed response return
jsonify({"error": "Service Unavailable"}), 503
```

try: verify_user_session(request) except Exception as e: # Log structured critical audit event
logger.critical(json.dumps({"event": "AUTH_FAIL", "error": str(e)})) # Enforce strict Fail-Closed response return
jsonify({"error": "Service Unavailable"}), 503

Fail-Open Risk

Unhandled exceptions default to bypass mode, inadvertently granting unauthorized administrative access.

Fail-Closed Defense

Revokes access immediately on failure, writes JSON audit logs, and returns HTTP 503 Service Unavailable.

THREAT ANALYSIS

STRIDE Threat Modeling Mapping

Section 6

Spoofing

Weak session auth mitigated via strict token validation.

Tampering

API parameter modification blocked by input sanitization.

Repudiation

Missing audit trails solved via structured JSON logging.

Info Disclosure

Verbose stack traces eliminated via A05 response hardening.

Denial of Service

Resource exhaustion handled with rate limiting controls.

Elevation of Privilege

Broken access control remediated with A10 fail-closed logic.

EXECUTION PLANNING

Feasibility & Risk Management

Section 4

Technical

Proficient in Python, Flask, Docker, and penetration testing.

Data & Tools

100% open-source software; zero data privacy concerns.

Time

Two-semester roadmap structured with bi-weekly sprints.

Economic

Zero financial cost utilizing local virtualization environments.

Risk Management Matrix

Risk Description	Likelihood	Mitigation Strategy
Docker local execution constraints	Medium	Reallocate local container runs; shift docs to cloud members.
ZAP scanner false positives	Low	Manual findings verification; scope scan rules strictly to A05/A10.

PROJECT TIMELINE

Two-Term Schedule & Agile Management

FIGURE 2: GANTT CHART SCHEDULE



Jira Sprint Tracking

GitHub Version Control

TEAM ACCOUNTABILITY

Division of Labor & Responsibilities

Section 8

Member Name	Role	Specific Tasks Owned	Phases	Workload
WONG Lai Man	**Team Leader**	Architecture, STRIDE Modeling, Proposal Integration, Gantt Schedule.	Phase 1–4	**100%**
KHAN M. MUSTAFA	**Developer / Tester**	Docker Deployment, A05/A10 Remediation, OWASP ZAP Scans.	Phase 2–3	**100%**
ZOUTAO	**Red Team Analyst**	A01-A10 Attack Methodology, Presentation Slides & Threat Diagrams.	Phase 1–4	**100%**

Declaration: Division of labor agreed upon by all team members. WONG Lai Man acts as designated Group Leader.

Expected Deliverables

Anticipated Accomplishments

Working POC System

Docker suite showcasing side-by-side vulnerable vs. secured endpoints.

Technical Docs

Full report with STRIDE threat models and remediation snippets.

Academic Artifacts

Completed Working Journals and 15–20 min live video demonstration.

Thank You! Q&A

We welcome any questions regarding our proposed solution and feasibility.